EXPERIMENT NO.3

To develop any distributed application using Message Passing Interface (MPI).

Title: To develop any distributed application using Message PassingInterface (MPI)

Objectives: To Design a distributed application using Message Passing Interface (MPI) for remote computationwhere client submits a string to the server and server returns the reverse of it to the client

Aim/ Problem Statement:: The scope of the assignment is to cover the following issues:

1. Client-Server communication implementation.
2. Request will be send between Client to Server for getting services from Server

Theory:

MPI (Message-Passing Interface) is a message-passing library interface specification. All parts of this definition are significant. MPI addresses primarily the message-passing parallel programming model, in which data is moved from the address space of one process to that of another process through cooperative operations on each process. Extensions to the \classical" message-passing model are provided in collective operations, remote-memory access operations, dynamic process creation, and parallel I/O. MPI is a significant, not an implementation; there are multiple implementations of MPI. This significant is for a library interface; MPI is not a language, and all MPI operations are expressed as functions, subroutines, or methods, according to the appropriate language bindings which, for C and Fortran, are part of the MPI standard. The standard has been defined through an open process by a community of parallel computing vendors, computer scientists, and application developers.

The goal of the Message-Passing Interface :

1. The major goal is to develop a widely used standard for writing message-passing programs.
2. Design an application programming interface (not necessarily for compilers or a system implementation library).

3. Allow efficient communication: Avoid memory-to-memory copying, allow over of computation and communication, and offered to communication co-processors, where available.
4. Allow for implementations that can be used in a heterogeneous environment.

An MPI program consists of autonomous processes, executing their own code, in an MIMD style. The codes executed by each process need not be identical. The processes communicate via calls to MPI communication primitives. Typically, each process executes in its own address space, although shared-memory implementations of MPI are possible. This document specifies the behaviour of a parallel program assuming that only MPI calls are used. The interaction of an MPI program with other possible means of communication, I/O, and process management is not specified. Unless otherwise stated in the specification of the standard, MPI places no requirements on the result of its interaction with external mechanisms that provide similar or equivalent functionality. This includes, but is not limited to, interactions with external mechanisms for process control, shared and remote memory access, _le system access and control, inter process communication, process signalling, and terminal I/O. High quality implementations should strive to make the results of such interactions intuitive to users, and attempt to document restrictions where deemed necessary. MPI provides the user with reliable message transmission. A message sent is always received correctly, and the user does not need to check for transmission errors, time-outs, or other error conditions. In other words, MPI does not provide mechanisms for dealing with failures in the communication system.

If the MPI implementation is built on an unreliable underlying mechanism, then it is the job of the implementor of the MPI subsystem to insulate the user from this unreliability, or to react unrecoverable errors as failures. Whenever possible, such failures will be reected as errors in the relevant communication call. Similarly, MPI itself provides no mechanisms for handling processor failures.

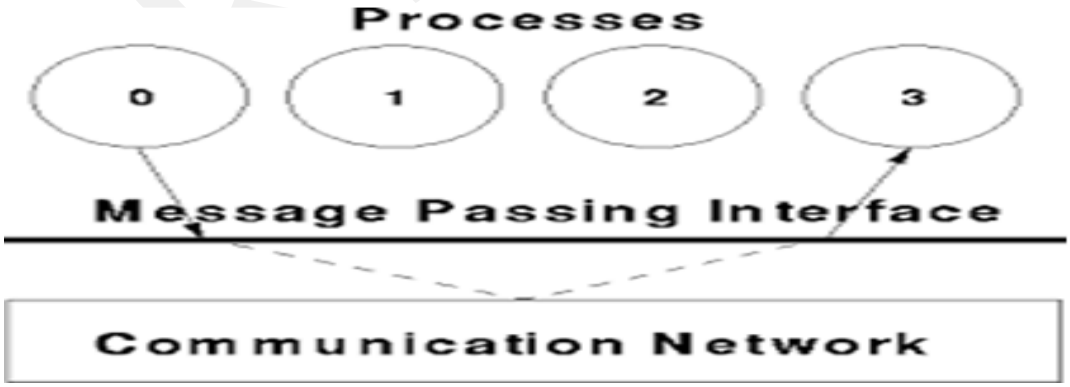


Figure: Message-Passing Interface Communication

- **MPI Data Type**

The Data Types available with MPI corresponding to c are listed in table 1.

Table 1: Predefined MPI data types corresponding to C data types

MPI Data Type	**C / Fortran Data Type**
MPI_CHAR	char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG_LONG_INT	signed long long int
MPI_LONG	signed long int
MPI_LONG_LONG	signed long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED_LONG	unsigned long int
MPI_UNSIGNED	unsigned int
MPI_FLOAT	float
MPI_DOUBLE	double

MPI Primitives:

Following are the some of primitives used in MPI for communication between the processes.

1. **MPI_Init :** Initialize or start MPI Communication
2. **MPI_Finalize:** Close or Exit MPI Communication
3. **MPI_Send() :** Mpi Blocking Send look like

send(dest, type, address, length)

Where:

- dest is an integer identi_er representing the process to receive the message.
- type is a nonnegative integer that the destination can use to selectively screen messages.
- Address describes a contiguous area in memory containing the message to be sent.

4. **MPI_Send() :** Mpi Blocking Receive look like

MPI_RECV (buf, count, datatype, source, tag, comm, status)

Where:

- buf initial address of receive buffer .
- count number of elements in receive buffer.
- datatype is a datatype of each receive buffer element
- source rank of source or MPI_ANY_SOURCE
- tag message tag or MPI_ANY_TAG
- comm communicator
- status status object

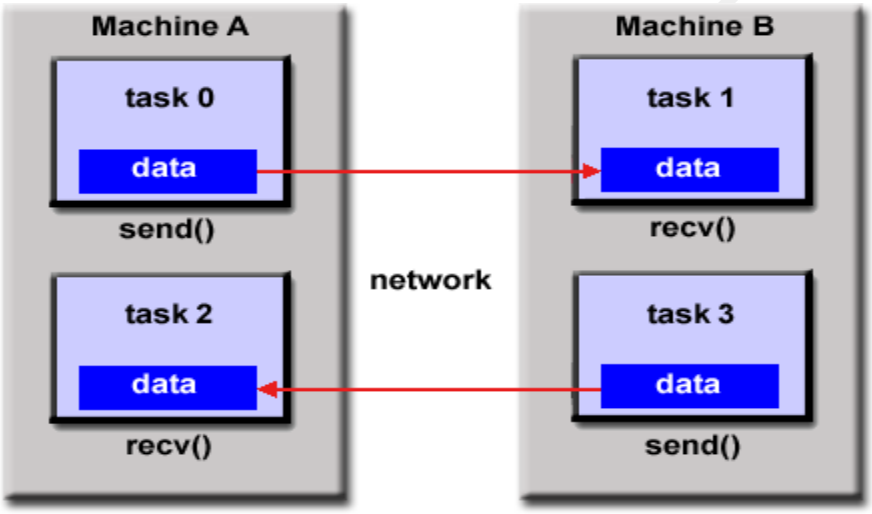


Figure: MPI Blocking Send and Receive Premitives.

Sending and receiving of messages by processes is the basic MPI communication mechanism. The basic point-to-point communication operations are send and receive. Their use is illustrated in the example below.

```
#include "mpi.h"
int main( int argc, char *argv[])
{
char message[20]; int myrank; MPI_Status status;
```

```
MPI_Init( &argc, &argv );
MPI_Comm_rank( MPI_COMM_WORLD, &myrank ); if (myrank == 0) /* code for process zero */
{
strcpy(message,"Hello, there");
MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process one */
{
MPI_Recv(message, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status); printf("received :%s:\n",
message);
}
MPI_Finalize(); return 0;
}
```

In this example, process zero (myrank = 0) sends a message to process one using the send operation MPI_SEND. The operation specifies a send buffer in the sender memory from which the message data is taken. In the example above, the send buffer consists of the storage containing the variable message in the memory of process zero. The location, size and type of the send buffer are specified
by the firrst three parameters of the send operation. The message sent will contain the 13 characters of this variable. In addition, the send operation associates an envelope with the message. This envelope specified the message destination and contains distinguishing information that can be used by the receive operation to select a particular message. The last three parameters of the send operation, along with the rank of the sender, specify the envelope for the message sent. Process one (myrank = 1) receives this message with the receive operation MPI_RECV. The message to be received is selected according to the value of its envelope, and the message data is stored into the receive buffer. In the example above, the receive buffer consists of the storage containing the string message in the memory of process one. The firrst three parameters of the receive operation specify the location, size and type of the receive buffer .The next three parameters are used for selecting the incoming message. The last parameter is used to return information on the message just received.

CONCLUSION:

We have studied the all methods of communication between two or group of processes using MPI (Message Passing Interface). We implemented Message Passing Interface communication program which shows us the effective and successful communication between the group of process.

FAQs:

5. What is MPI?
6. What is are the Goals of MPI?
7. What are the Primitives of MPI?
8. What is Use of Blocking Send in MPI?
9. What is Use of Blocking Receive in MPI?
10. What are Data Types in MPI?
11. Write a Program in C for MPI Group communication?

RSCOE-IT